

Variables en Python

Capítulo técnico, educativo y práctico

Curso Curso de Python	Módulo / Unidad 1
Capítulo Variables	Nivel Desde principiante hasta nivel práctico
Enfoque Desarrollo de Software Python	Meta Comprender, practicar y aplicar variables

Una guía completa para entender cómo un programa representa información, cómo asignarla, reutilizarla, actualizarla y diseñarla con claridad profesional.

1. Objetivo del capítulo

Este capítulo desarrolla las variables desde el modelo mental más sencillo hasta un uso práctico en programas. El objetivo no es memorizar una definición: es aprender a identificar qué información necesita un programa y representarla con nombres claros y valores adecuados.

Al finalizar este capítulo serás capaz de:

- Explicar qué es una variable, por qué existe y qué problema resuelve.
- Crear variables con nombres válidos y descriptivos.
- Representar enteros, decimales, texto y estados booleanos.
- Actualizar valores mediante reasignación y usar variables en cálculos.
- Detectar errores frecuentes de sintaxis y diseño.
- Modelar datos de máquinas, sensores, inventarios, empleados y sistemas de producción.
- Preparar información que posteriormente utilizarán operadores, condicionales, bucles y funciones.

Resultado esperado: dado un problema pequeño, podrás decidir qué datos necesita el programa, elegir nombres apropiados y construir asignaciones correctas.

2. Introducción: ¿qué son las variables?

Un programa necesita trabajar con información. Una aplicación puede recordar un nombre; un videojuego necesita conservar una puntuación; un sistema industrial puede manejar una temperatura, una velocidad, un límite de seguridad y el estado de un motor. Esa información no siempre es fija: cambia mientras el programa se ejecuta.

Una **variable** es un nombre que permite referirse a un valor y reutilizar esa información posteriormente. Como analogía inicial, puedes imaginar una etiqueta que identifica una caja. La etiqueta permite localizar el contenido; técnicamente, Python trabaja con nombres asociados a objetos y valores.

¿Por qué existe?

Sin variables tendríamos que repetir valores constantemente y sería difícil representar cambios. Una variable permite dar significado a un dato, reutilizarlo y actualizarlo. El nombre `temperatura_actual` comunica qué representa el valor; el número aislado `78.5` no lo hace por sí solo.

¿Dónde se utiliza?

- Aplicaciones empresariales: clientes, totales y estados.
- Software industrial: sensores, motores, producción y alarmas.
- Videojuegos: posición, energía, puntuación y nivel.
- Automatización: límites, lecturas y estados de equipos.
- Cualquier programa que necesite conservar, transformar o comunicar información.

3. El modelo mental: nombre, asignación y valor

La forma general de crear o actualizar una variable es:

```
nombre_variable = valor
```

Parte	Función
nombre_variable	Identificador que utilizarás después para acceder al dato.
=	Operador de asignación.
valor	Información o resultado que será asociado al nombre.

Python evalúa la expresión situada a la derecha y después realiza la asignación al nombre de la izquierda.

```
temperatura = 25
nombre = "Ana"
motor_encendido = True
```

Representación simplificada:

```
temperatura ■■■■■■■■ 25
nombre      ■■■■■■■■ "Ana"
motor_encendido ■■■■■■ True
```

Esta representación es un modelo de aprendizaje. Más adelante, al estudiar objetos, identidad y mutabilidad, podrás profundizar en las referencias que utiliza Python.

4. Ejemplo básico y explicación línea por línea

```
edad = 25
nombre = "Ana"
altura = 1.65
es_estudiante = True

print(nombre)
print(edad)
```

- `edad = 25`: almacena un entero.
- `nombre = "Ana"`: almacena una cadena de texto.
- `altura = 1.65`: almacena un decimal.
- `es_estudiante = True`: almacena un valor booleano.
- `print(nombre)` utiliza el valor asociado al nombre, no la palabra escrita entre comillas.

Resultado esperado: Ana y 25 en líneas separadas.

Error típico: `print("edad")` imprime el texto edad. Para imprimir el contenido se escribe `print(edad)`.

5. Tipos de datos y tipado dinámico

Un **tipo de dato** indica qué clase de información representa un valor. Python utiliza tipado dinámico: normalmente no escribes el tipo al crear una variable; el tipo pertenece al valor u objeto que el nombre representa.

Tipo	Ejemplo	Uso típico
int	cantidad = 50	Contadores y cantidades enteras.
float	temperatura = 23.7	Mediciones y valores decimales.
str	modelo = "X200"	Texto, nombres y códigos.
bool	activo = True	Estados y resultados verdadero/falso.

```
nivel = 87
print(type(nivel))
```

`type()` permite inspeccionar el tipo durante el aprendizaje o la depuración. No es necesario utilizarlo para cada variable en un programa normal.

Tipado dinámico no significa diseño arbitrario

```
dato = 10
dato = "diez" # Permitido por Python
```

Python permite reasignar un nombre a otro tipo de valor, pero en código mantenible conviene que una variable conserve un propósito coherente.

6. Reasignación y actualización

```
piezas = 0
piezas = piezas + 1
print(piezas)
```

Paso a paso: primero `piezas` vale 0. Después Python evalúa la parte derecha, obtiene 1 y reasigna ese resultado al mismo nombre. La salida es 1.

```
piezas = 0
piezas += 1
```

La segunda forma es una actualización abreviada. Es útil para contadores y valores acumulativos.

Símbolo	Significado	Ejemplo
=	Asignación	temperatura = 30
==	Comparación de igualdad	temperatura == 30

7. Nombres de variables: reglas y convenciones

- Un nombre puede contener letras, números y el carácter `_`.
- No puede comenzar con un número.
- No puede contener espacios.
- Python distingue mayúsculas y minúsculas.
- No debe ser una palabra reservada del lenguaje.

Válido	Inválido	Motivo
temperatura_actual	2temperatura	No puede comenzar con número.
sensor_1	sensor actual	No se permiten espacios.
totalProduccion	for	for es palabra reservada.
_contador	temperatura-max	El guion medio no es válido.

Convención recomendada: snake_case

Para variables, es habitual usar minúsculas y separar palabras con guiones bajos: `cantidad_piezas`, `velocidad_actual`, `limite_temperatura`. La consistencia mejora la lectura y el mantenimiento.

Elige nombres que representen el concepto, no solo el tipo. `temperatura_actual` suele comunicar más que `dato_float`.

8. Ejemplos progresivos

Nivel 1 — Comprender el concepto

```
nombre = "Luis"
edad = 20
print(nombre)
print(edad)
```

El programa crea dos variables y muestra sus valores. Es el ejemplo mínimo para distinguir entre un nombre y el contenido asociado.

Nivel 2 — Integrar variables y operadores

```
piezas_por_caja = 24
cantidad_cajas = 15
total_piezas = piezas_por_caja * cantidad_cajas

print("Total de piezas:", total_piezas)
```

Primero se guardan datos originales; después se calcula un resultado y se guarda en una variable nueva. Esta separación facilita comprender el cálculo.

8. Ejemplos progresivos (continuación)

Nivel 3 — Caso práctico: máquina industrial

```
nombre_maquina = "CNC-01"
temperatura_actual = 78.5
temperatura_maxima = 85.0
motor_activo = True

margen_temperatura = temperatura_maxima - temperatura_actual

print("Máquina:", nombre_maquina)
print("Temperatura:", temperatura_actual)
print("Margen:", margen_temperatura)
print("Motor activo:", motor_activo)
```

Cada variable representa una parte del estado del sistema. El programa todavía no toma decisiones; eso será trabajo de los condicionales. Sin embargo, ya prepara la información que necesitarán esas decisiones.

Nivel 4 — Ejemplo profesional: reporte de producción

```
linea_produccion = "Línea A"
piezas_planificadas = 1200
piezas_producidas = 1140
piezas_defectuosas = 18

piezas_buenas = piezas_producidas - piezas_defectuosas
piezas_pendientes = piezas_planificadas - piezas_producidas

print("Reporte de producción")
print("Línea:", linea_produccion)
print("Piezas buenas:", piezas_buenas)
print("Piezas pendientes:", piezas_pendientes)
```

- `linea_produccion` identifica el contexto.
- `piezas_planificadas` representa la meta.
- `piezas_producidas` conserva el registro original.
- `piezas_buenas` y `piezas_pendientes` son resultados derivados.
- Separar conceptos evita perder información y facilita ampliar el programa.

Errores comunes en este ejemplo: usar el mismo nombre para producción total y producción buena; calcular antes de asignar los datos necesarios; sobrescribir una variable original que todavía se necesita.

9. Cómo funciona: flujo visual

1. Dato 78.5	2. Asignación <code>temperatura = 78.5</code>	3. Uso <code>temperatura + 1</code>	4. Resultado 79.5
------------------------	---	---	-----------------------------

En programas reales este proceso puede repetirse: llega una lectura, una variable se actualiza, se realizan cálculos y otros componentes utilizan el estado más reciente.

10. Comparaciones y errores de confusión

Concepto	Qué es	Diferencia principal
Variable	Nombre asociado a información.	Permite reutilizar y representar un concepto.
Valor literal	Dato escrito directamente.	No posee un nombre reutilizable.
=	Asignación.	Guarda o actualiza.
==	Comparación.	Pregunta si dos valores son iguales.
"texto"	Cadena literal.	Con comillas es texto; sin ellas puede ser un nombre.

Confusión frecuente: `Total` y `total` no son el mismo nombre. Python distingue mayúsculas y minúsculas.

11. Errores comunes al trabajar con variables

Usar un nombre antes de asignarlo

Ejemplo: `print(temperatura)` antes de crear la variable. Normalmente producirá un `NameError`. Corrige el orden de las instrucciones y revisa errores de escritura.

Olvidar comillas en texto

`modelo = CNC01` intenta utilizar un nombre llamado CNC01. Para almacenar texto usa `modelo = "CNC01"`.

Usar espacios

`nivel tanque = 80` es inválido. Usa `nivel_tanque`.

Comenzar con un número

`1sensor` es inválido. Usa `sensor1`.

Confundir mayúsculas

Crear `Total` y utilizar `total` produce nombres diferentes.

Sobrescribir accidentalmente

Reutilizar `total` para un número y después para un texto destruye el significado anterior.

Nombres ambiguos

`x`, `a` o `dato` dificultan el mantenimiento cuando el programa crece.

- Define el propósito del dato antes de elegir el nombre.
- Mantén una convención consistente.
- Conserva datos originales cuando todavía serán necesarios.
- Lee el código como si otra persona tuviera que modificarlo.
- Prueba el programa con distintos valores y observa las salidas.

12. Buenas prácticas

- **Usa nombres descriptivos:** reducen ambigüedad y mejoran el mantenimiento.
- **Mantén snake_case:** la consistencia hace que el código sea más fácil de recorrer.
- **Una variable, un propósito:** evita que un mismo nombre represente conceptos diferentes.
- **Usa variables intermedias cuando aclaren un cálculo:** una operación visible suele ser más fácil de revisar.
- **Incluye unidades cuando sea importante:** temperatura_celsius o velocidad_m_s pueden evitar errores técnicos.
- **Evita imprimir información sensible en depuración:** en software real, no todos los datos deben aparecer en registros o consola.
- **Prioriza claridad antes que microoptimizaciones:** para variables básicas, un diseño correcto y legible suele ser más valioso que ahorrar unas pocas líneas.

13. Conectando conocimientos

Concepto	Relación con variables
print()	Muestra valores almacenados.
Operadores	Transforman valores y producen resultados.
Entrada	Los datos recibidos suelen guardarse en variables.
Condicionales	Toman decisiones usando valores.
Bucles	Actualizan contadores y procesan estados repetidamente.
Funciones	Reciben y crean variables locales; devuelven resultados.
Estructuras de datos	Organizan múltiples valores relacionados.

Las variables son una base: el aprendizaje continúa con operadores, entrada, condiciones, bucles, funciones y estructuras de datos. Los conceptos nuevos reutilizan lo que ya aprendiste.

14. Ejercicios prácticos

No se proporciona una solución inmediata. Intenta diseñar primero la información y después escribir el código.

Ejercicio 1 — Comprensión básica

Objetivo: crear cuatro variables para tu nombre, edad, altura y estado de estudiante. Requisitos: usa nombres descriptivos y muestra todos los valores. Pista: identifica el tipo de cada dato.

Ejercicio 2 — Aplicación

Contexto: una máquina tiene código, temperatura, velocidad y estado. Requisitos: guarda cada dato en una variable y crea un reporte. Restricción: no uses nombres de una sola letra.

Ejercicio 3 — Integración

Una caja contiene cierta cantidad de piezas y existen varias cajas. Crea variables para ambos datos y una tercera para el total. Pista: conserva separados los datos originales y el resultado.

Ejercicio 4 — Caso práctico

Representa producción planificada, producción real y piezas defectuosas. Calcula piezas buenas y pendientes. Decide qué variables deben conservarse y cuáles son resultados.

Ejercicio 5 — Reto

Diseña la parte inicial de un monitor para un tanque industrial. Representa tanque, nivel actual, mínimo, máximo, temperatura, estado de bomba y ciclos. Reglas: nombres descriptivos, tipos apropiados y al menos dos valores calculados. Pistas: primero lista los datos originales; después identifica cálculos; finalmente diseña el reporte.

15. Mini proyecto: Registro de una estación de producción

Contexto

Una estación necesita generar un reporte básico. No se conectará todavía a sensores reales: el objetivo es construir un modelo de información mediante variables.

Objetivo

Representar datos de producción, realizar cálculos sencillos y presentar resultados legibles.

Requisitos funcionales

- Identificador de estación.
- Objetivo de piezas y piezas producidas.
- Piezas defectuosas.

- Temperatura actual y límite recomendado.
- Estado del sistema.
- Cálculo de piezas buenas y pendientes.
- Reporte final claro.

Requisitos técnicos

- Nombres descriptivos.
- snake_case.
- Uso apropiado de int, float, str y bool.
- Conservar datos originales.
- Evitar nombres ambiguos.

Paso 1

Antes de escribir código, lista la información que existe en el problema y decide qué variable representará cada elemento.

Paso 2

Agrega variables derivadas para los resultados. No sobrescribas datos que todavía necesites.

Paso 3

Construye el reporte y comprueba que una persona externa entienda cada etiqueta.

Paso final

Cambia algunos valores iniciales y verifica que todos los resultados se actualicen correctamente. Después revisa los nombres y mejora los que no comuniquen claramente su propósito.

16. Comprueba tus conocimientos

- 1. Define variable con tus propias palabras.
- 2. ¿Qué problema resuelve una variable?
- 3. Diferencia entre nombre y valor.
- 4. ¿Qué significa `=` en `total = 10`?
- 5. Resultado de `piezas = 5`; `piezas = piezas + 2`.
- 6. ¿Cuál es inválido: `sensor_1`, `1sensor`, `sensorUno`?
- 7. ¿Por qué `temperatura_actual` es mejor que `x`?
- 8. ¿Qué muestra `nombre = "Ana"; print(nombre)`?
- 9. ¿Qué muestra `print("nombre")` y por qué?
- 10. Diferencia entre `=` y `==`.
- 11. Tipo de `25`, `18.5`, `"Robot"` y `False`.
- 12. ¿Qué es una reasignación?
- 13. Detecta el error: `velocidad actual = 10`.
- 14. Analiza: `dato = 20`; `dato = "error"`.
- 15. Propón tres variables para un empleado.
- 16. Propón cuatro variables para un motor industrial.
- 17. ¿Por qué separar `piezas_producidas` y `piezas_buenas`?
- 18. ¿Cómo preparan las variables el uso de condicionales y bucles?

17. Respuestas y explicaciones

1. Un nombre que permite referirse a un valor y reutilizar esa información.
2. Representar datos que pueden utilizarse y actualizarse sin repetir valores por todo el código.
3. El nombre es el identificador; el valor es la información representada.
4. Asignación: el resultado de la derecha se asocia al nombre de la izquierda.
5. 7, porque se suma 2 al valor actual y el resultado se reasigna.
6. 1sensor; no puede comenzar con un número.
7. Porque comunica el significado del dato.
8. Ana, porque nombre se evalúa como la variable.
9. La palabra nombre, porque las comillas crean una cadena literal.
10. = asigna; == compara igualdad.
11. int, float, str y bool, respectivamente.
12. Asignar un nuevo valor a un nombre existente.
13. Los nombres no pueden contener espacios; por ejemplo, velocidad_actual.
14. Es permitido, pero cambia el tipo y el propósito del nombre; puede reducir claridad.
15. Ejemplo: nombre_empleado, horas_trabajadas, empleado_activo.
16. Ejemplo: motor_id, velocidad_rpm, temperatura_celsius, motor_activo.
17. Representan conceptos distintos y conservar el dato original permite otros cálculos.
18. Los condicionales evalúan valores y los bucles pueden actualizar o procesar esos valores repetidamente.

18. Resumen del capítulo

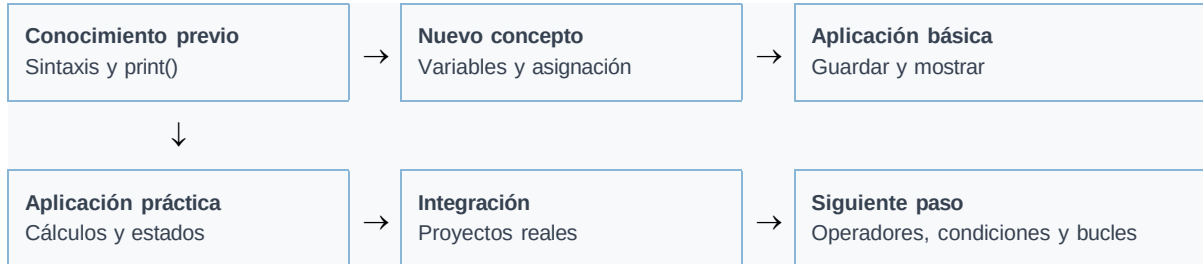
Concepto	Para qué sirve	Cuándo utilizarlo	Error común
Variable	Representar información.	Cuando un dato necesita significado y reutilización.	Usarla antes de asignarla.
=	Asignar o actualizar.	Al crear o modificar estado.	Confundirlo con ==.
Tipo	Representar correctamente un dato.	Al modelar información.	Confundir texto, números o booleanos.
Reasignación	Actualizar estado.	Cuando cambia un valor.	Sobrescribir datos necesarios.
snake_case	Mejorar legibilidad.	Al nombrar variables.	Usar espacios o nombres ambiguos.

Ideas clave que debes recordar

- Una variable asocia un nombre con un valor.
- = asigna y == compara.
- Python utiliza tipos como int, float, str y bool.

- Una variable puede actualizarse mediante reasignación.
- Los nombres deben ser válidos, descriptivos y consistentes.
- snake_case mejora la legibilidad.
- Las variables representan conceptos del problema, no solo números o texto.
- Separar datos originales de resultados derivados mejora la claridad.
- Las variables alimentan cálculos, decisiones, bucles y funciones.

Mapa de aprendizaje



Las variables son una herramienta fundamental porque permiten que el programa tenga información con significado. El siguiente paso consiste en transformar esos valores, obtenerlos desde otras fuentes y utilizarlos para tomar decisiones y controlar el flujo del programa.